IMAGE PROCESSING -Preparation-

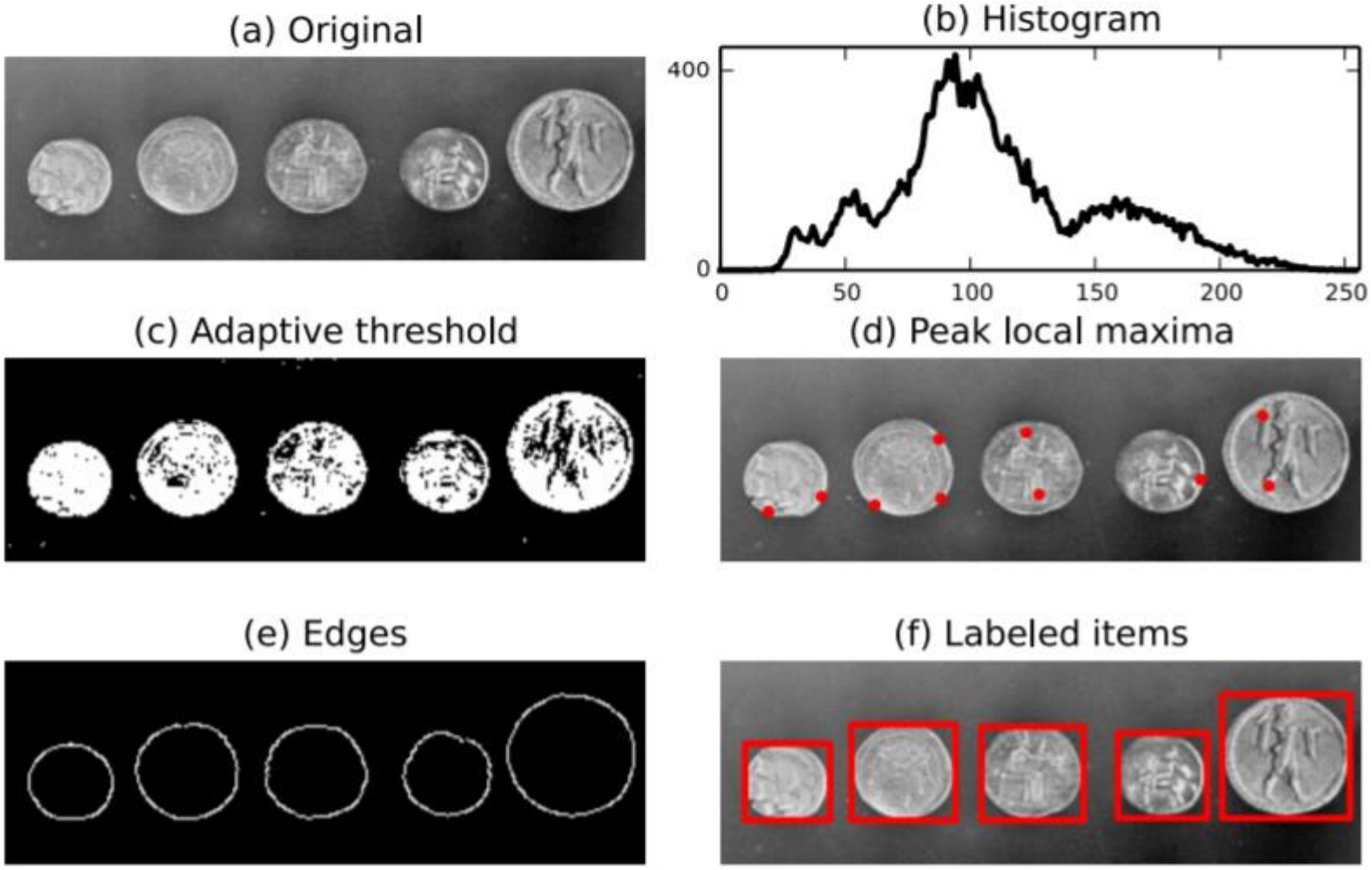
Nada Fitrieyatul Hikmah

Requirements

- Install Python 3.6
- Scikit-image ➔ 0.15
- Numpy ➔ 1.12
- Scipy ➔ 1.0
- Matplotlib ➔ 2.1
- Jupyter Notebook ➔ 4.0
- Scikit-learn ➔ 0.18

Scikit-image

- Scikit-image is an image processing library that implements algorithms, provides a well-documented API in the Python programming language.



Installing scikit-image

- Go to : **https://scikit-image.org/**

`scikit-image` comes pre-installed with several Python distributions, including Anaconda, Python(x,y) and WinPython.

On all major operating systems, install it via shell/command prompt:

```
pip install scikit-image
```

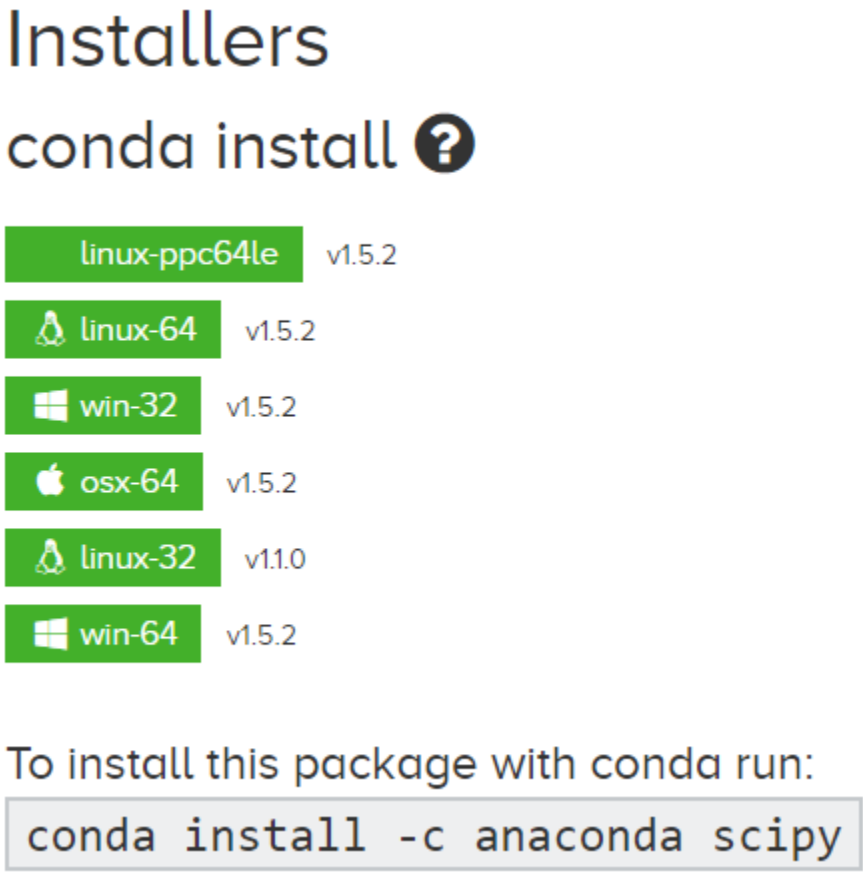
If you are running Anaconda or miniconda, use:

```
conda install -c conda-forge scikit-image
```

The wheels can be downloaded manually from PyPI.

Installing SciPy

- SciPy is a Python-based ecosystem of open-source software for mathematics, science, and engineering.
- Open : https://anaconda.org/anaconda/scipy



Installing Matplotlib

- Matplotlib is a python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms.
- Open : https://anaconda.org/anaconda/matplotlib

Install Numpy

- Go to : https://numpy.org/install/
- The only prerequisite for NumPy is Python itself. If you don't have Python yet and want the simplest way to get started, we recommend you use the Anaconda Distribution - it includes Python, NumPy, and other commonly used packages for scientific computing and data science.

CONDA

If you use `conda`, you can install it with:

```
conda install numpy
```

Installing Scikit-learn

- Scikit-learn is a free software machine learning library for the Python programming language. It features various classification, regression and clustering algorithms including support vector machines, random forests, gradient boosting, k-means and DBSCAN, and is designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy.
- Go to : https://scikit-learn.org/stable/install.html

Scikit-learn requires:

- Python (>= 2.6 or >= 3.3),
- NumPy (>= 1.6.1),
- SciPy (>= 0.9).

If you already have a working installation of numpy and scipy, the easiest way to install scikit-learn is using `pip`

```
pip install -U scikit-learn
```

or `conda`:

```
conda install scikit-learn
```

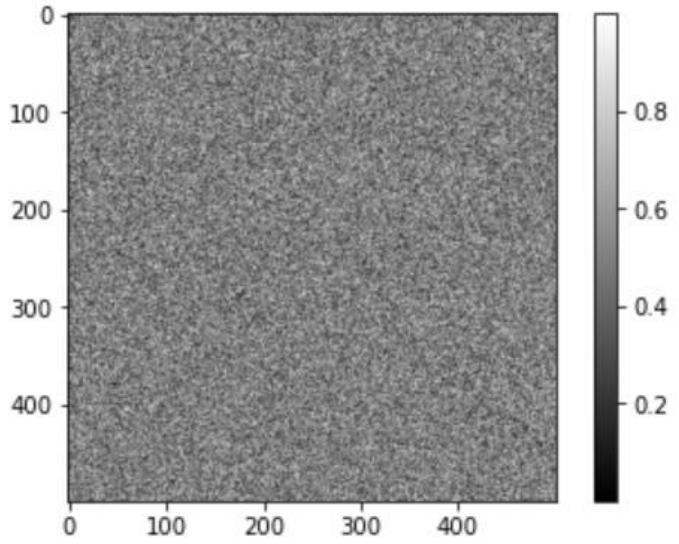
Image are Arrays

Nada Fitrieyatul Hikmah

Images are numpy arrays

- Images are represented in scikit-image using standard numpy arrays.
- The simplest image we can make is a 2-dimestional matrix :

In [10]:
```
import numpy as np
from matplotlib import pyplot as plt

random_image = np.random.random([500,500])

plt.imshow(random_image,cmap='gray')
plt.colorbar();
```

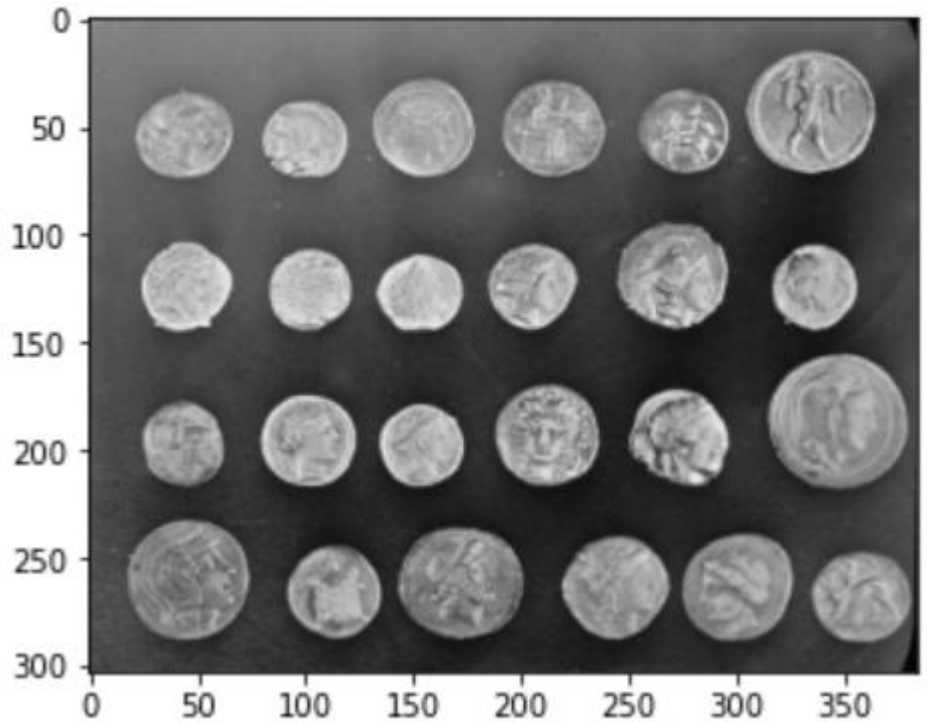


“Real-world” images

Standard test image module from skimage : https://scikit-image.org/docs/dev/api/skimage.data.html

```
In [11]: from skimage import data

         coins = data.coins()

         print('Type:', type(coins))
         print('dtype:', coins.dtype)
         print('shape:', coins.shape)

         plt.imshow(coins, cmap='gray');
```

```
Type: <class 'numpy.ndarray'>
dtype: uint8
shape: (303, 384)
```



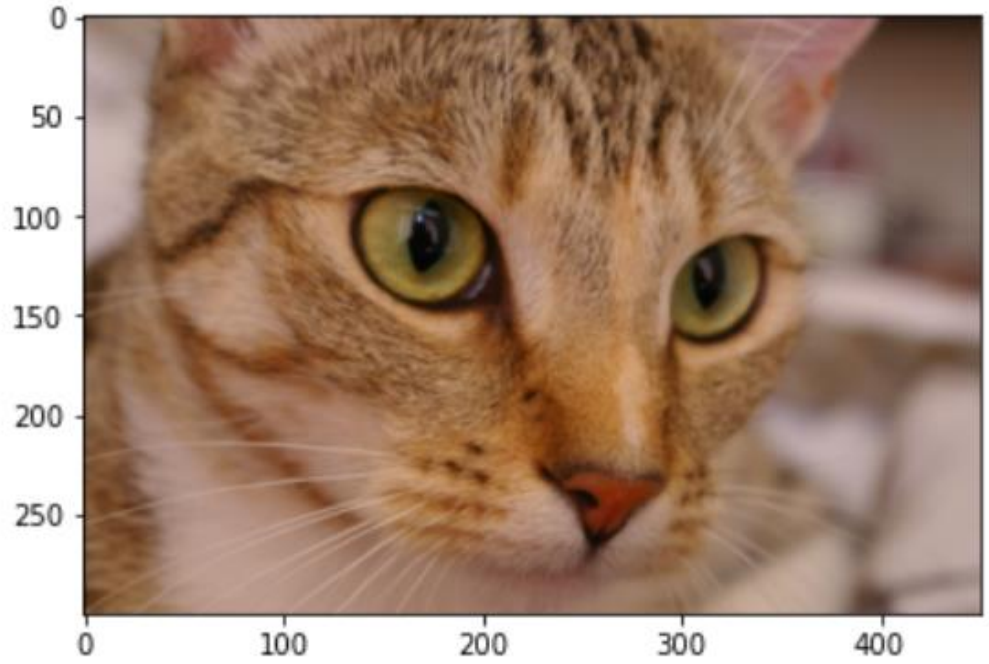
Next question is….

How we represent color arrays??

What would we need in order to represent a color image??

In [12]:
```
cat = data.chelsea()
print("Shape:", cat.shape)
print("Values min/max:", cat.min(), cat.max())

plt.imshow(cat);
```

```
Shape: (300, 451, 3)
Values min/max: 0 231
```



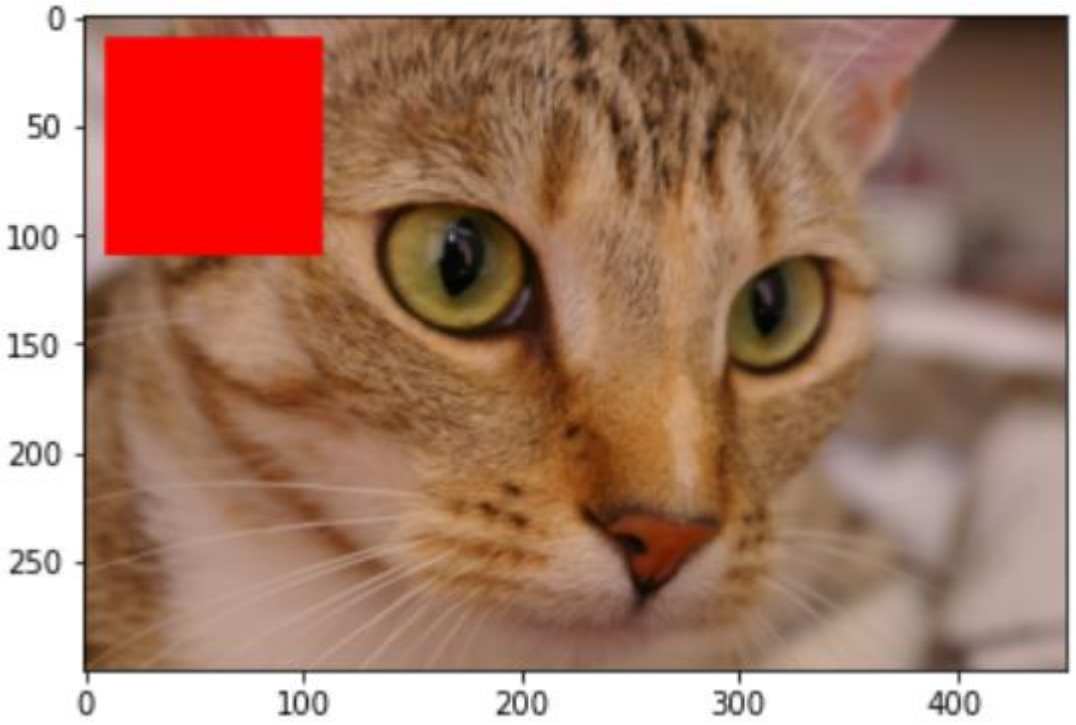
Color Image

- A color image is a 3D array, where the last dimension has size 3 and represents the red, green, and blue channels.

Image just Numpy arrays

- These are *just NumPy arrays*. E.g., we can make a red square by using standard array slicing and manipulation:

```
cat[10:110, 10:110] = [255, 0, 0]  # [red, green, blue]
plt.imshow(cat);
```



Displaying images using matplotlib

```
1  from skimage import data
2
3  img0 = data.chelsea()
4  img1 = data.rocket()
```

```
1  import matplotlib.pyplot as plt
2
3  f, (ax0, ax1) = plt.subplots(1, 2, figsize=(20, 10))
4
5  ax0.imshow(img0)
6  ax0.set_title('Cat', fontsize=18)
7  ax0.axis('off')
8
9  ax1.imshow(img1)
10 ax1.set_title('Rocket', fontsize=18)
11 ax1.set_xlabel(r'Launching position $\alpha=320$')
12
13 ax1.vlines([202, 300], 0, img1.shape[0], colors='magenta', linewidth=3, label='Side tower position')
14 ax1.plot([168, 190, 200], [400, 200, 300], color='white', linestyle='--', label='Side angle')
15
16 ax1.legend();
```

For more on plotting, see the Matplotlib documentation and pyplot API.

Data Types and Image Values (1)

- In literature, one finds different conventions for representing image values:

```
0 - 255   where  0 is black, 255 is white
0 - 1     where  0 is black, 1 is white
```

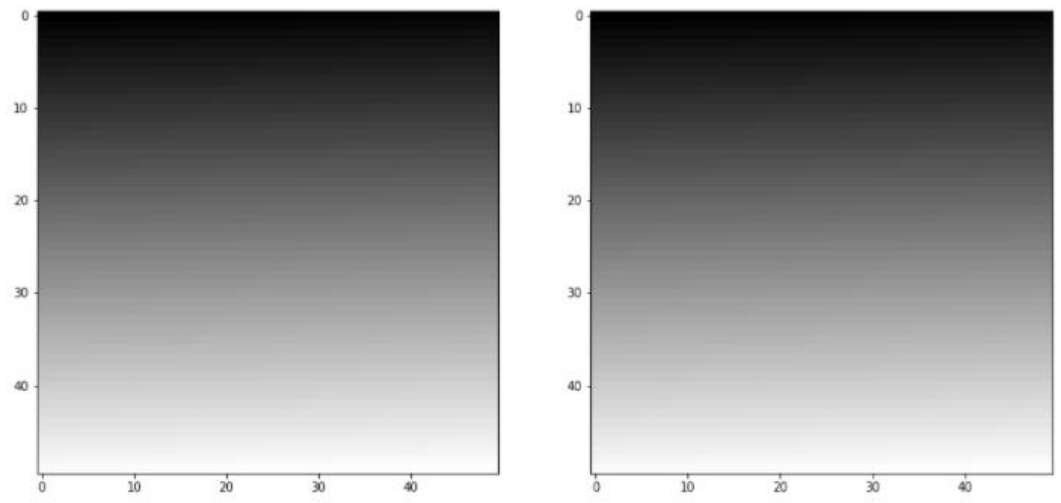
- scikit-image supports both conventions--the choice is determined by the data-type of the array.

Data Types and Image Values (2)

- Example 1 : I generate two valid images

```
1  linear0 = np.linspace(0, 1, 2500).reshape((50, 50))
2  linear1 = np.linspace(0, 255, 2500).reshape((50, 50)).astype(np.uint8)
3
4  print("Linear0:", linear0.dtype, linear0.min(), linear0.max())
5  print("Linear1:", linear1.dtype, linear1.min(), linear1.max())
6
7  fig, (ax0, ax1) = plt.subplots(1, 2, figsize=(15, 15))
8  ax0.imshow(linear0, cmap='gray')
9  ax1.imshow(linear1, cmap='gray');
```

```
Linear0: float64 0.0 1.0
Linear1: uint8 0 255
```



The library is designed in such a way that any data-type is allowed as input, as long as the range is correct (0-1 for floating point images, 0-255 for unsigned bytes, 0-65535 for unsigned 16-bit integers).

Image I/O (1)

- Mostly, we won't be using input images from the scikit-image example data sets. Those images are typically stored in JPEG or PNG format. Since scikit-image operates on NumPy arrays, any image reader library that provides arrays will do. Options include imageio, matplotlib, pillow, etc.
- scikit-image conveniently wraps many of these in the io submodule, and will use whichever of the libraries mentioned above are installed:

```
1  from skimage import io
2
3  image = io.imread('D:\Teaching\Pengolahan Citra Medika\Database\Brain.jpeg')
4
5  print(type(image))
6  print(image.dtype)
7  print(image.shape)
8  print(image.min(), image.max())
9
10 plt.imshow(image);
```

```
<class 'numpy.ndarray'>
uint8
(630, 630, 3)
0 255
```

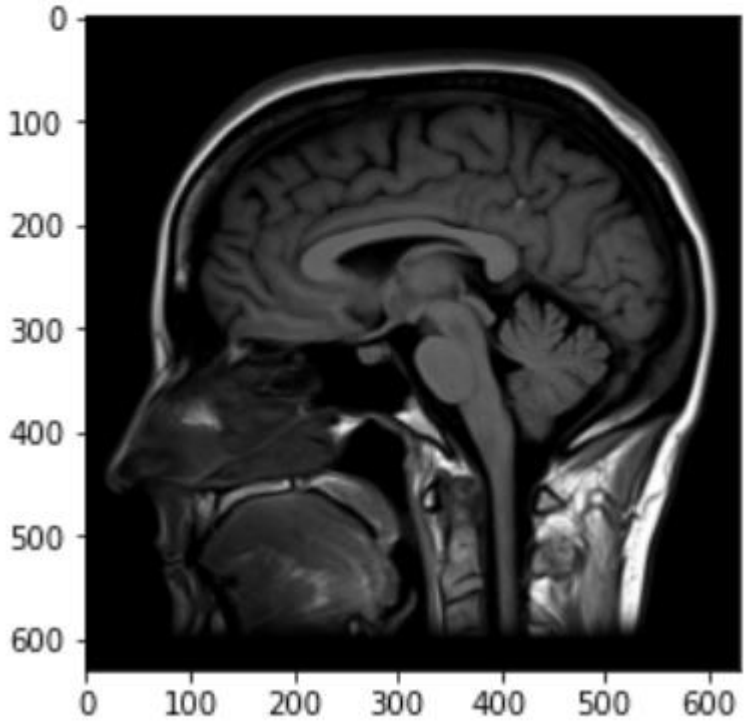


Image I/O (2)

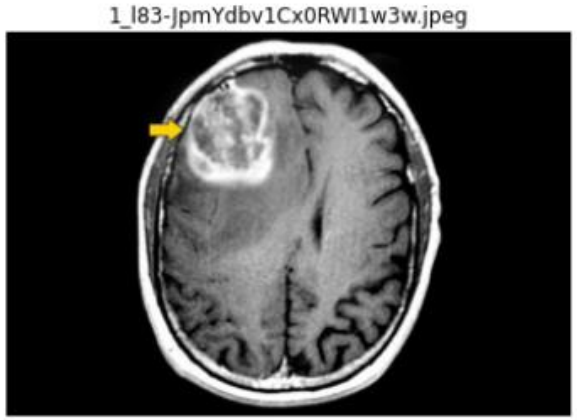
- We also have the ability **to load multiple images** or multi-layer TIFF (Tag Image File Format) images ➔is called **image collection.**

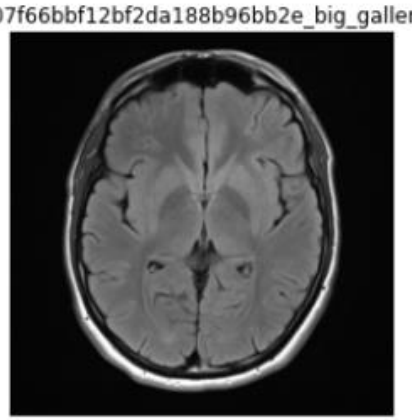
```
1  ic = io.ImageCollection('D:\Teaching\Pengolahan Citra Medika\Database\*.jpeg')
2  print('Type:', type(ic))
3
4  ic.files
```

```
Type: <class 'skimage.io.collection.ImageCollection'>

['D:\\Teaching\\Pengolahan Citra Medika\\Database\\1_l83-JpmYdbv1Cx0RWI1w3w.jpeg',
 'D:\\Teaching\\Pengolahan Citra Medika\\Database\\78de707f66bbf12bf2da188b96bb2e_big_gallery.jpeg',
 'D:\\Teaching\\Pengolahan Citra Medika\\Database\\Brain.jpeg']
```

Image I/O (3)

```python
import os

f, axes = plt.subplots(nrows=3, ncols=len(ic) // 1, figsize=(20, 10))

# Gunakan `axes.ravel()` untuk mengubahnya menjadi daftar
axes = axes.ravel()

for ax in axes:
    ax.axis('off')

for i, image in enumerate(ic):
    axes[i].imshow(image, cmap='gray')
    axes[i].set_title(os.path.basename(ic.files[i]))

plt.tight_layout()
```





Note : Enumerate?

- Enumerate gives us each element in a container, along with its position.

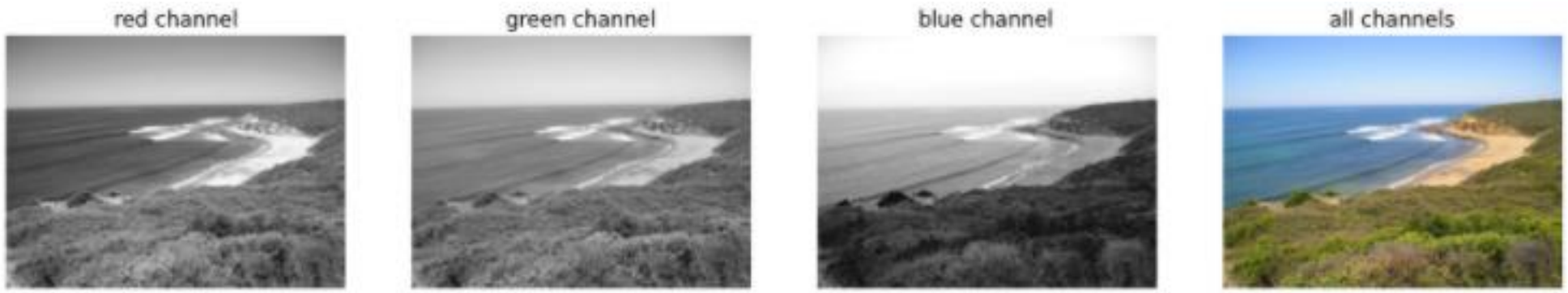
```
1 BMEs = ['Intelligent Biomedical Instrumentation', 'Assistive Technology and Rehabilitation Engineering', 'Medical Imaging an
2 for i, BME in enumerate(BMEs):
3     print('Expertise in BME consist of {} is {}'.format(i, BME))
```

```
Expertise in BME consist of 0 is Intelligent Biomedical Instrumentation
Expertise in BME consist of 1 is Assistive Technology and Rehabilitation Engineering
Expertise in BME consist of 2 is Medical Imaging and Image Processing
Expertise in BME consist of 3 is Medical Informatics
```

Visualizing RGB Channels (1)

- Display the different color channels of the image along (each as a gray-scale image).

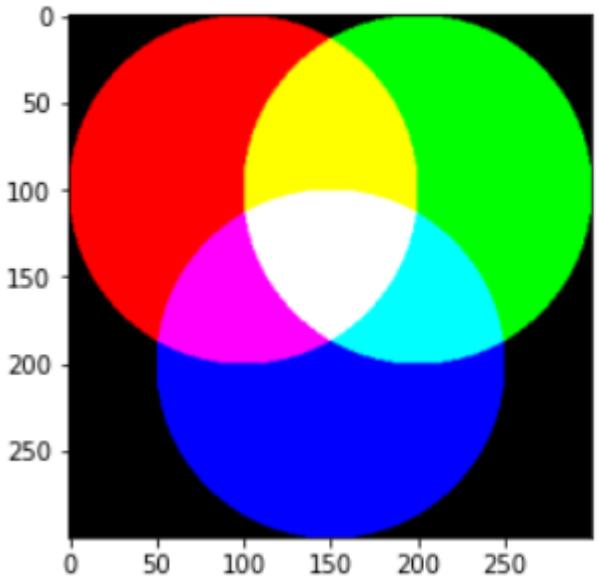
```python
# --- baca citra ---

image = plt.imread('D:\Teaching\Pengolahan Citra Medika\Database\Beach.jpg')

# --- tetapkan setiap channel warna ke variabel yang berbeda ---

r = image[:, :, 0]
g = image[..., 1 ]
b = image[..., 2]

# --- Tampilkan citra dari channel R, G, B ---

f, axes = plt.subplots(1, 4, figsize=(16, 5))

for ax in axes:
    ax.axis('off')

(ax_r, ax_g, ax_b, ax_color) = axes

ax_r.imshow(r, cmap='gray')
ax_r.set_title('red channel')

ax_g.imshow(g, cmap='gray')
ax_g.set_title('green channel')

ax_b.imshow(b, cmap='gray')
ax_b.set_title('blue channel')

# --- Disini, kita buat tumpukan layer RGB ---
#     untuk membentuk citra berwarna ---
ax_color.imshow(np.stack([r, g, b], axis=2))
ax_color.set_title('all channels');
```



Visualizing RGB Channels : what is color combination?

```python
from skimage import draw
import numpy as np

red = np.zeros((300, 300))
green = np.zeros((300, 300))
blue = np.zeros((300, 300))

r, c = draw.circle(100, 100, 100)
red[r, c] = 1

r, c = draw.circle(100, 200, 100)
green[r, c] = 1

r, c = draw.circle(200, 150, 100)
blue[r, c] = 1

f, axes = plt.subplots(1, 3)
for (ax, channel) in zip(axes, [red, green, blue]):
    ax.imshow(channel, cmap='gray')
    ax.axis('off')
```

```python
color_stack = np.dstack([red, green, blue])
plt.imshow(color_stack);
```



Exercise : Convert to Grayscale

- The *relative luminance* of an image is the intensity of light coming from each point. Different colors contribute differently to the luminance: it's very hard to have a bright, pure blue, for example. So, starting from an RGB image, the luminance is given by:

$$Y = 0.2126R + 0.7152G + 0.0722B$$

- Use Python matrix multiplication, **@**, to convert an RGB image to a grayscale luminance image according to the formula above.

- Compare your results to that obtained with skimage.color.rgb2gray.